Per-Tenant Rate Limiter

Token buckets in a hash map, run on a real production trace

Tracks: Concurrency and hashing (token bucket, lazy refill, a lock per key) **Code:** `src/TenantRateLimiter/`. **Tests:** 24, run with `dotnet test`. **Demo:** `demo/`, on the Azure LLM inference trace

---

0. The problem

> An LLM API serves many customers. Each one gets a limit on requests per minute and on input tokens per minute. Short bursts are fine, but one customer's burst must never slow down anyone else. A request over the limit gets a 429 and is told how long to wait.

▼ Hint 1
What do you need to remember per customer? Does it need a timer?

▼ Hint 2
A request fits under the request limit but not the token limit. What should it take?

▼ Hint 3
Two requests from the same customer arrive on two threads at the same moment. What can go wrong?

---

1. Background

- The Claude API "uses the token bucket algorithm to do rate limiting. This means that your capacity is continuously replenished up to your maximum limit, rather than being reset at fixed intervals." Limits are "set at the organization level" and measured in requests, input tokens and output tokens per minute. "Short bursts of requests can exceed the limit and trigger rate limit errors." (Claude API rate limits)
- On the same API, "ITPM rate limits count only uncached input tokens". So a cached prompt prefix (see Prefix-cache radix tree) also leaves more room under the limit.
- Azure OpenAI checks request rates over "a small period of time, typically 1 or 10 seconds", so "rate limiting occurs for a 600-RPM deployment if more than 10 requests are received during each 1-second period" (Azure OpenAI quota).
- .NET ships `TokenBucketRateLimiter` and `PartitionedRateLimiter`, which splits traffic by a key such as "user ID, IP address, or API key" (ASP.NET Core rate limiting).

---

2. The mental model

A bucket holds at most 5 tokens and gets 1 back every second. Each request takes 1. No token, no entry.

Time	Before	Requests	Passed	429	After
0	5 (starts full)	3	3	0	2

Time	Before	Requests	Passed	429	After
0	2	3	2	1	0
1	0 + 1 = 1	0			1
4	1 + 3 = 4	0			4
10	min(5, 4 + 6) = 5	7	5	2	0

Two settings, two jobs: **capacity** caps the biggest burst, and the **refill rate** sets the long-run average.

No timer is needed. When a request arrives, work out what came back since last time:

```
tokens = min(capacity, tokens + seconds passed x refill rate)
wait   = (cost - tokens) / refill rate          // the Retry-After
```

For input tokens, a request costs its prompt size instead of 1. A per-tenant limiter is a hash map from customer to two buckets, one for requests and one for input tokens.

---

3. Ways to limit

Method	Memory per customer	Bursts
Fixed window (count per clock minute)	One counter	Up to 2x the limit in a few seconds around the reset
Sliding window log (a timestamp per request)	Grows with traffic	Exact
Sliding window counter (two windows, weighted)	Two counters	Approximate
Token bucket	**Two numbers: tokens and last refill time**	**At most the capacity at once**
Leaky bucket (a queue drained at a fixed rate)	The queue	Smoothed out rather than let through

These are the standard definitions; only the token bucket and the fixed window are built here. A test shows the fixed window case: limit 5 per minute, 5 requests at 0:59 and 5 at 1:00. The fixed window lets all 10 through. The token bucket lets 5 through, because one second only brings back 5/60 of a token.

---

4. Why it's correct

Lazy refill gives the same answers as refilling step by step. The bucket only grows between requests, and capping once at the end is the same as capping at every step. A test runs 300 random traces against a bucket that refills in steps of 1/1024 s. Both versions must agree exactly, request by request.

Both or neither. A request is checked against both buckets before it takes from either. If it took a request token and then failed on input tokens, the customer would lose a request for nothing.

One lock per customer. Checking and taking must be one step: without the lock, two threads can both see 1 token left and both take it. Customers don't share locks, so one busy customer never makes another wait. A

test sends 20,000 requests across threads at one instant and exactly the bucket's 1,000 get through.

Time never runs backwards. A thread can read the clock and then get the lock after a thread that read it later. The bucket ignores the older time instead of moving its clock back.

Removing idle customers is safe. A full bucket gives the same answers as a new one. So once both of a customer's buckets are full again, the entry can be deleted. If a request found the entry just before it was deleted, it sees the deleted flag and fetches the new one. A test runs cleanup and requests together and never sees extra requests get through.

---

5. On real traffic: the Azure LLM inference trace

Inputs, and where each comes from:

Input	Value	Source
Traffic	Two traces recorded over the same hour, coding (8,819 requests) and conversation (19,366), replayed at their recorded times	Azure LLM Inference Dataset 2023, CC-BY; its paper is Splitwise (Patel et al., ISCA 2024)
Customers	Each trace is one customer of the same API	**Our setup.** The traces are two services, not two customers
Limits	Twice each customer's own average, for both requests and input tokens	**Our choice**
Bucket size	60 seconds of the limit, then 10 seconds for comparison	**Our choice**
After a 429	Not retried, only counted	**Our choice**

The traffic:

	Coding	Conversation
Average	154.0 requests/min, 315,371 input tokens/min	331.8 requests/min, 383,158 input tokens/min
Limit (2x)	308.0 requests/min, 630,742 input tokens/min	663.7 requests/min, 766,316 input tokens/min
Busiest 60 seconds	723 requests	522 requests
Busiest 1 second	72 requests	19 requests

Results (from `demo/`, reproduce with section 8):

Bucket holds	Customer	Allowed	429	Longest Retry-After
60 s	Coding	8,501	318 (3.61%)	0.7 s
60 s	Conversation	19,366	0	
10 s	Coding	6,499	2,320 (26.31%)	0.7 s
10 s	Conversation	19,366	0	

What the real trace shows:

- **Same rule, different result.** Both customers get twice their average. The bursty one gets cut, the steady one never does.
- **The bucket size matters as much as the rate.** The same per-minute limit with a 10-second bucket rejects about 7 times as many coding requests (2,320 against 318). This is the "429 while under my per-minute limit" that both API docs warn about.
- **No customer slows another.** Conversation replayed on its own gets the same answer for every one of its 19,366 requests.
- **The waits are short.** The bucket refills all the time instead of resetting each minute, so no wait can be longer than one big prompt's worth of refill: 7,437 tokens at 630,742 / 60 = 10,512 tokens a second is 0.7 seconds. The longest seen was 0.7 seconds.

---

6. Assumptions and limits

	Status
Bucket maths, both or neither, one lock per customer, cleanup	**Tested** (24 tests, incl. 300 random traces against a step-by-step bucket, and thread races)
Same answers as .NET's `TokenBucketRateLimiter`	**Tested** for bursts at one instant and for requests bigger than the bucket. In our check, its manual refill (`TryReplenish`) added nothing until real time had passed, so it can't be stepped through time in a test
Token bucket, per organization, requests and input tokens per minute	**Cited** (section 1)
Two customers, the limits, the bucket sizes, no retries	**Our choices** (section 5)
Output tokens per minute	**Left out.** A separate limit; the output size isn't known when the request arrives
Cached tokens	**Left out.** The trace has no cache data
Many servers	**Left out.** This limiter lives in one process. With 3 servers each keeping its own count, a customer gets 3x the limit (ASP.NET Core docs); one shared limit needs a shared store

---

7. Tests

Area	**What they protect**
Bucket	the section 2 table step by step; a bigger cost waits for the difference; a cost above capacity never fits; an older time is ignored; bad settings
Against a step-by-step bucket	300 random traces, exact agreement on every decision and every token count
Against .NET	a burst of mixed costs at one instant; a cost above capacity is refused up front
Per customer	one customer's burst doesn't touch another; both or neither; Retry-After is the longer of the two waits; too big takes nothing; a shorter bucket
Threads	20,000 requests at one instant, exactly 1,000 through; cleanup running during requests lets nothing extra through
Fixed window	10 through around the reset against 5 for the token bucket

Area	What they protect
Trace	CSV parsing and bad rows; per-minute averages; busiest window; replay per customer; the same answers alone or next to a noisy customer

8. Run it

The trace files aren't in this repo. Download `AzureLLMInferenceTrace_code.csv` and `AzureLLMInferenceTrace_conv.csv` from the dataset page.

```
cd dsa/concurrency/per-tenant-rate-limiter
dotnet test
dotnet run --project demo/TenantRateLimiter.Demo -- --code <path>/AzureLLMInferenceTrace_code.csv --conv
<path>/AzureLLMInferenceTrace_conv.csv
```

Requires .NET 10 SDK.

9. Common mistakes

Mistake	What goes wrong
A timer per customer to add tokens	Thousands of timers doing nothing; work it out when a request arrives
No cap on the refill	A quiet customer saves up and fires an enormous burst
Taking from one bucket before checking the other	A rejected request still uses up the customer's allowance
Check, then take, without a lock	Two threads both take the last token
`DateTime.Now` as the clock	It can jump; use a monotonic clock
Making a too-big request wait	It never fits; reject it straight away
Never removing customers	The map grows forever

Sources

- Anthropic: Claude API rate limits
- Microsoft: Azure OpenAI quota
- Microsoft: Rate limiting middleware in ASP.NET Core
- Microsoft .NET: TokenBucketRateLimiter.TryReplenish
- Microsoft Azure: Azure LLM Inference Dataset 2023, CC-BY. Data not included here.
- Patel et al. (2024), "Splitwise: Efficient generative LLM inference using phase splitting", ISCA: arXiv:2311.18677. The dataset page asks users to cite this paper.